

5-minute demo script

AI Revenue Recovery Agent — Razorpay Buildathon, Track 03

All figures below are from the **800-case, seed 42** batch. Reproduce with `POST /batch/run?n=800&seed=42`, or the **800 cases** option in the dashboard.

Before you hit record

```
# terminal 1
cd backend && uvicorn app.main:app --port 8000

# terminal 2
cd frontend && npm run dev
```

Open **http://localhost:5173**, leave the size on **800 cases**, press **Run batch**, and wait for it to finish. Record with the batch already loaded — never film a spinner.

Close other tabs. Zoom to ~110% so type survives compression. The whole demo is one browser tab.

0:00 – 0:35 · The problem

"A Razorpay merchant loses revenue four different ways — a card payment fails, a customer abandons checkout, a subscription auto-debit bounces, an invoice goes past due. In this batch that's **₹1.35 crore at risk across 800 cases**.

The usual answer is a retry cron: try the card again, send an SMS, try again. That does recover some of it. My question was — how much of what it recovers is actually the **agent's** doing, and how would you ever prove it?"

Screen: top of the dashboard, "What this agent does" visible.

0:35 – 1:25 · What the agent is

Point at each of the five stage cards as you name it.

"Detect — Razorpay webhooks and batch cases normalise into one shape, so the live path and the demo path run identical code.

Diagnose — the first decision that matters. **88.6% of failures are classified by a deterministic rule engine**, not a model. Free, instant, auditable. The LLM is only asked about the free-text tail the rules can't match, and below 70% confidence it returns UNKNOWN and the case goes to a human. I don't guess with money.

Decide — stopping rules, then the playbook, then compliance, in that order.

Act — payment links, mandate retries, Hinglish messages, on a schedule.

Observe — outcomes close the loop.

Every number on those cards is live from the run that just finished."

1:25 – 2:40 · One case, end to end ← the most important minute

Click **Walk through one case**.

"One failure. The customer's card **expired**. The rule engine mapped Razorpay's `card_expired` reason at 100% confidence.

Now watch what that fact **does**." — point at "**Why this constrains the options**" — "An expired card isn't unlikely to work. It is **guaranteed** to decline. So retrying the stored card **doesn't exist as a step** in this playbook. The ladder goes straight to a new instrument."

Point at the decision trail.

"Detected, diagnosed — then **COMPLY**. It was the middle of the night, so the agent **deferred** the message to 9am instead of cancelling it. That's the difference between a compliance rule and lost revenue: it kept both.

Then it acted, and the money came back."

Point at the baseline strip.

"Same customer, naive policy. It retried the expired card — **scored zero point zero percent**. Not unlikely. Impossible.

Across the batch that's **card-expired at 49.2% against 13.6%**."

2:40 – 3:40 · Why the numbers are real

Scroll to the verdict panel.

"This is the part I most want you to push on.

A recovery rate has no counterfactual in it. So **every case runs twice** — once under my policy, once under a naive dunning baseline — against the **same customers**, drawing the **same random numbers**. A paired experiment. Any difference is policy, not luck.

28.9% against 19.8%. ₹46.95 lakh recovered against ₹34.37 lakh. That's **₹12.59 lakh incremental**, with a 90% confidence interval of ₹3.52 lakh to ₹21.95 lakh — the interval clears zero. The agent won **122 cases** the baseline lost and lost **49** the baseline won.

And it did that with **883 customer contacts against 1,298** — more money, from fewer messages."

Scroll to "Where the lift comes from".

"Here's where it comes from — and where it doesn't.

Insufficient funds: 25.5% against 8.5%. The instrument works; the balance doesn't. The baseline re-presents five minutes later into the same empty account. Mine defers the retry to the next likely salary-credit date.

Subscriptions: 46.3% against 7.3% — same mechanism on auto-debit.

But look at the bottom of that list. **Gateway timeouts — 80.3% against the baseline's 86.4%. My agent is worse.** A transient fault just needs an immediate retry, and the dumb cron does that better than my backoff does. Same with UPI collect. I'm showing you the rows where I lose.

And **risk-blocked and disputed invoices — zero in both columns.** That's correct. Re-presenting a risk-declined payment is what card-scheme rules exist to prevent. The compliant recovery rate is zero, and encoding that is the feature."

3:40 – 4:20 · Guardrails and the audit trail

Scroll to the compliance panel.

"Quiet hours, per-channel consent, a three-contacts-per-week cap, a cost ceiling so it won't spend ₹45 of a human's time chasing a ₹300 cart, and a ₹50,000 autonomy ceiling on consumer collections.

489 actions deferred to a legal window, 113 blocked outright, zero violations. The baseline, on identical inputs, would have committed **1,854**.

And it stops. Every case ends as recovered, escalated, suppressed or exhausted — never silently dropped. **₹36.61 lakh went to humans, and I count all of it as not recovered."**

Point at the green badge in the header.

"Every decision is in a hash-chained audit log — each entry commits to the hash of the one before it. That badge is a live verification of **16,922 entries**. Edit any historical row and it turns red and names the sequence number where the trail diverges. A test proves it by mutating a row."

4:20 – 5:00 · Real, simulated, and what broke

"Being precise about this matters more than the headline.

Real: webhook ingestion with HMAC signature verification, the rule engine, Payment Link creation against Razorpay's test API, the compliance layer, the stopping rules, the audit chain.

Simulated: whether a synthetic customer actually pays. There is no real customer in a batch. Those base rates are plausible assumptions, not measured constants. What makes the comparison hold is that **both policies are scored by the same model on the same customers** — so the **difference** survives the assumptions even where an absolute number wouldn't.

What broke — two things. My ₹50,000 autonomy ceiling was applying a consumer-collections rule to B2B invoices, so the agent escalated every receivable while the baseline chased them. It **lost by ₹1.06 lakh**. I only caught it because the baseline arm existed.

And a subtler one: my simulator replenished bank balances at a uniformly random time. But my agent's whole insufficient-funds strategy is to retry on payday — and if money arrives at random, targeting payday is worth nothing. The lift I'd measured there was an artefact. I made the simulated world reflect how salary actually lands, and the strategy started earning its result for the right reason.

70 tests, including one that greps the pipeline to prove the agent can't read the simulator's ground truth. Thank you."

Delivery notes

- **Do not rush 1:25–2:40.** The expired-card walkthrough is the pitch. Everything else is evidence for it.
- Say "**zero point zero percent**" slowly. It's the line they'll remember.
- **Volunteer the losing rows.** Gateway timeouts and UPI collect are where you lose, and saying so first is worth more than the headline. Judges are looking for whether you know where your own numbers are soft.
- If asked about the LLM: it's wired, tested and currently **off** — no credits on the key — so 91 ambiguous cases route to humans instead of being guessed. That's the designed degradation, and the dashboard reports it honestly rather than hiding it.
- Don't read this on camera. Learn the six headings and talk.

The six headings

1. The problem — ₹1.35 crore at risk
2. What the agent is — five stages
3. One case — expired card, 0.0%
4. Why it's real — paired baseline, ₹12.59 lakh, CI clears zero
5. Guardrails — 489 deferred, 0 violations, 16,922 audit entries
6. Real vs simulated, and what broke